BarbTech

Documentação técnica de segurança, LGPD e robustez

Tarefa 5 [AC2] · Entrega final de Segurança da Informação

Threat modeling · vulnerabilidades e correções · implementação de cada requisito, com prints · plano de evolução · roteiro de demonstração

Vinicius Bueno e demais integrantes da squad [preencher]
[Disciplina] · [Instituição] · 2026

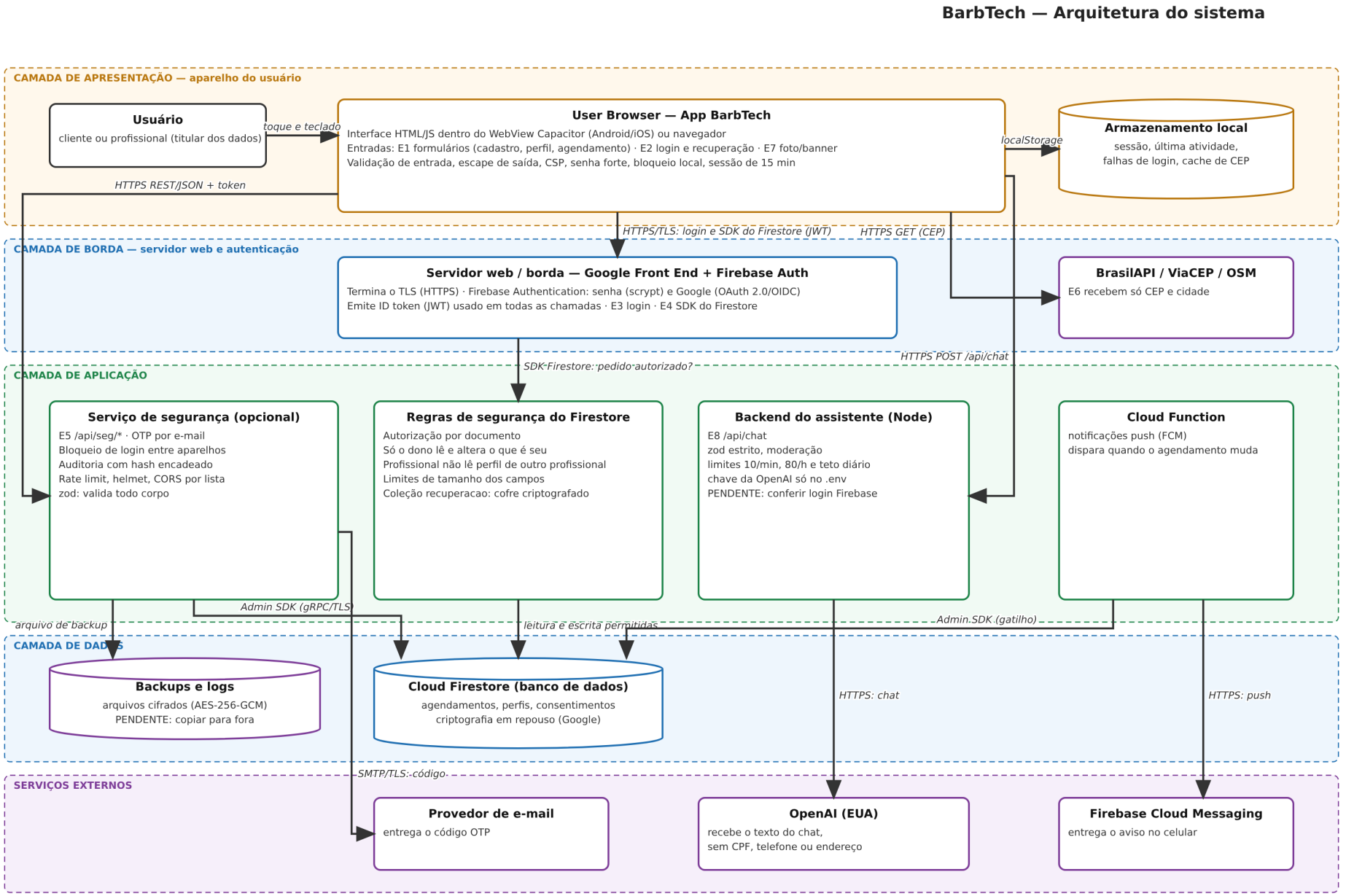
1. Como ler este documento

O BarbTech é um aplicativo móvel de agendamento para barbearias (Capacitor, com interface em HTML e JavaScript) que fala diretamente com o Firebase Authentication e o Cloud Firestore. Há dois serviços Node opcionais: o backend do assistente de IA (pasta *backend-chatbot*) e o serviço de segurança (pasta *backend-seguranca*), que entrega o OTP por e-mail, o bloqueio de login entre aparelhos, a auditoria e o backup. Este texto segue a ordem da Tarefa 5: o modelo de ameaças, as vulnerabilidades encontradas com as correções, cada requisito técnico com sua evidência, o plano de evolução e um roteiro para a apresentação.

Todas as evidências vêm de execuções reais do código em ambiente simulado: o Firebase é substituído por um módulo simulado servido ao navegador, e os servidores Node rodam de verdade. Não houve teste com o Firebase real, com envio real de e-mail, com as regras do Firestore no emulador nem com o aplicativo em um celular. Cada seção diz o que o código garante e o que depende de configuração no console do Firebase. O Firebase guarda senhas com scrypt, uma função de derivação de chave com uso intenso de memória, da mesma classe do bcrypt e do Argon2; o bcrypt é usado pelo serviço de segurança para o OTP e para os códigos de recuperação do servidor.

Resumo do que a squad entrega. Threat modeling em diagrama (Figuras 1 e 2) e em texto; 15 vulnerabilidades corrigidas e 8 pendências assumidas (seção 3); hash e criptografia (seção 4.1); limite de requisições, expiração de sessão e bloqueio (4.2); validador visual de senha e MFA com aplicativo autenticador (4.3); OTP por e-mail (4.4); logs, backup e hardening (4.5); erros genéricos (4.6); política de privacidade e consentimento (4.7); e plano de evolução (seção 5).

Figura 1 – Arquitetura do BarbTech em camadas, com fluxos nomeados e pontos de entrada E1 a E8



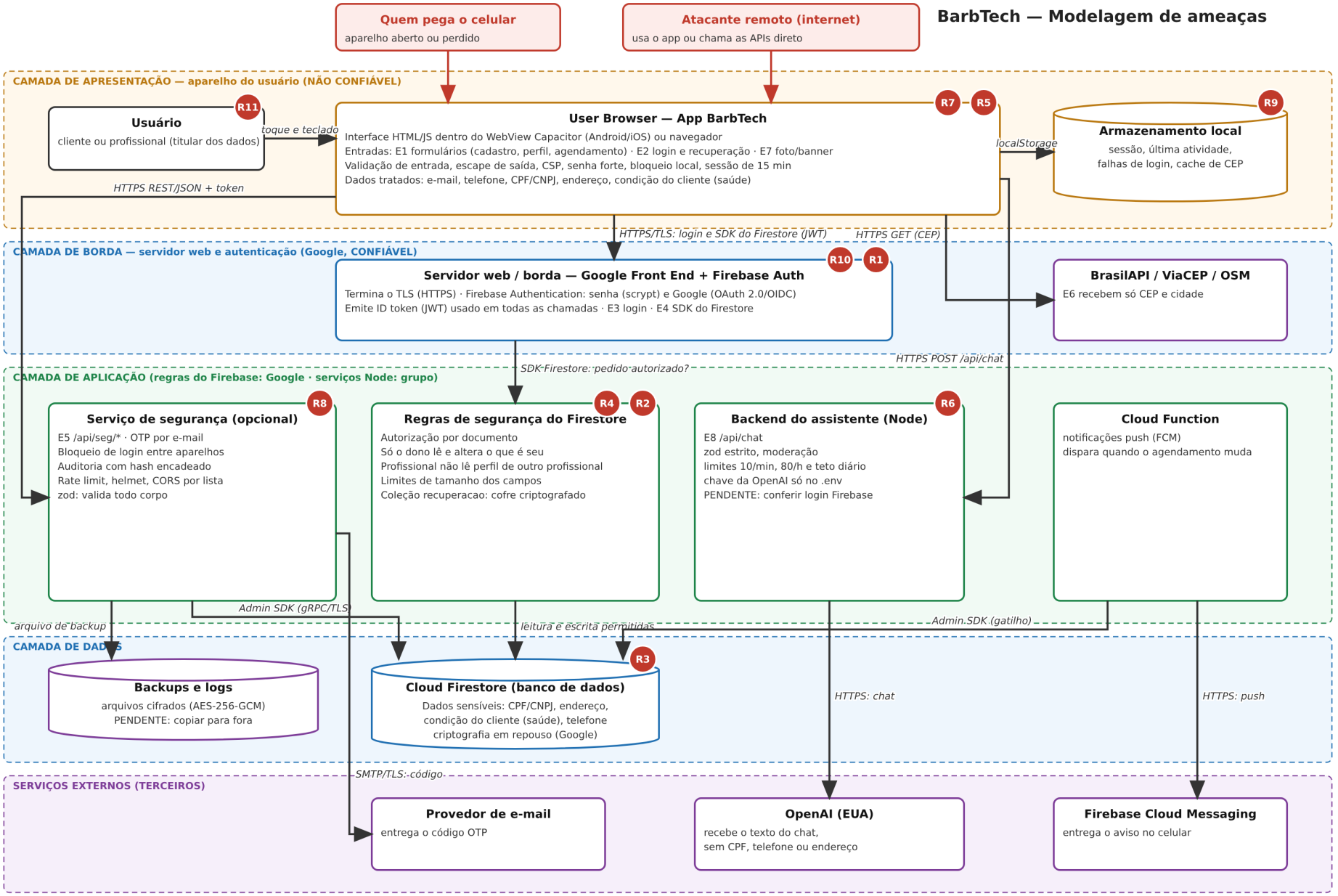
Seta cinza = fluxo de dados, com o protocolo no rótulo · E1 a E8 = pontos de entrada de dados
As camadas seguem o modelo navegador → servidor web → aplicação → banco de dados; no BarbTech o servidor web é a borda do Google/Firebase.

Fonte: elaborado pelo grupo no draw.io (arquivo BarbTech-Arquitetura-ThreatModel.drawio).

2. Threat modeling

A Figura 1 mostra a arquitetura e a Figura 2 a modelagem de ameaças, construída sobre ela. O método foi o STRIDE aplicado a cada componente e a cada fluxo, no draw.io. O diagrama define as fronteiras de confiança: o aparelho do usuário é não confiável, a infraestrutura do Google (autenticação e regras do Firestore) é confiável e gerenciada, e os servidores próprios e os serviços de terceiros exigem controles próprios. Os dados sensíveis (CPF, CNPJ, endereço e a condição de saúde do cliente) estão marcados no banco, e os pontos de entrada, de E1 a E8, estão nomeados na arquitetura.

Figura 2 – Modelagem de ameaças: atacantes, fronteiras de confiança, dados sensíveis e riscos R1 a R11



Seta cinza = fluxo de dados · seta vermelha = ataque · círculo R1 a R11 = risco da análise · cada camada indica se é confiável
R1 força bruta · R2 controle de acesso · R3 dados pessoais · R4 recuperação de conta · R5 XSS · R6 assistente de IA · R7 dados do cliente adulterados
R8 dependências · R9 aparelho perdido · R10 indisponibilidade · R11 LGPD (governança)

Fonte: elaborado pelo grupo no draw.io (arquivo BarbTech-Arquitetura-ThreatModel.drawio).

Os onze riscos do diagrama, com a análise completa de impacto, probabilidade, STRIDE, LGPD, CWE e CVSS, estão no Relatório Técnico (documento separado). Em uma frase cada: **R1** força bruta e roubo de credenciais; **R2** falha no controle de acesso do banco; **R3** exposição de dados pessoais e de saúde; **R4** tomada de conta pela recuperação; **R5** injeção de script na tela (XSS); **R6** abuso do backend do assistente; **R7** adulteração de dados calculados no aparelho; **R8** dependências vulneráveis; **R9** aparelho perdido; **R10** indisponibilidade; **R11** governança da LGPD. Pela pontuação, a ordem de adequação é R1, R11, R8, R6, R7, R2, R3, R4, R9, R10, R5.

3. Vulnerabilidades encontradas e correções aplicadas

Esta seção lista o que foi encontrado nas versões anteriores e o que foi feito. Cada item diz como era, o que mudou e como verificar. As pendências estão no fim, sem maquiagem.

3.1 Corrigidas

Senha fraca aceita. A versão 5 aceitava qualquer senha de 6 caracteres. Agora o app exige 8 ou mais caracteres, com maiúscula, minúscula, número e caractere especial (máximo de 128), com validador visual a cada tecla. O servidor de segurança repete a regra no OTP. Verificar: Figura 3.

Tentativas de login ilimitadas. Não havia bloqueio. Agora cinco senhas erradas bloqueiam o acesso, por 5, 15 e 60 minutos no aparelho (a cada bloqueio novo) e por 15 minutos no servidor, valendo para qualquer aparelho. Verificar: Figuras 4 e 5.

Sessão eterna. Quem abria o app ficava logado para sempre. Agora a sessão cai após 15 minutos sem toque, com aviso um minuto antes, e também se o app ficar fechado além desse prazo.

Erros técnicos mostrados ao usuário. A função de erro exibia o código e um trecho da mensagem do Firebase. Agora só aparecem frases em português escritas pelo grupo; o detalhe vai apenas para o console do desenvolvedor. Login com e-mail inexistente e com senha errada dão a mesma mensagem. Verificar: Figuras 4 e 14.

Recuperação só por link e sem confirmação extra. Foi criada a recuperação por código de 6 dígitos (OTP) enviado ao e-mail, com validade de 10 minutos, uso único e 5 tentativas, mantendo o link do Firebase como alternativa. Também existe o código único de recuperação do cadastro, que funciona sem servidor. Verificar: Figuras 8 e 9.

Ausência de segundo fator. Foi implementado o MFA do Firebase Authentication com aplicativo autenticador (código TOTP de 6 números, RFC 6238), com QR code, janela de código no login e desativação. Quem exige o segundo fator é o servidor do Google, e não o aplicativo. Verificar: Figuras 6 e 7.

Nenhum registro de auditoria e nenhum backup. Foram criados o log de auditoria, com hash encadeado e e-mail apenas como HMAC, e a rotina diária de backup criptografado com AES-256-GCM e retenção de 14 dias. Verificar: Figuras 10 e 11.

Impossível excluir dados. As regras do Firestore proibiam exclusão, o que impede o direito de eliminação. Agora o titular apaga perfil, área privada, agenda, banner, consentimento e o cofre de recuperação. O app tem o botão correspondente e a exportação dos dados.

Sem política de privacidade e sem consentimento. Foram criadas a tela de política, acessível sem login, o aceite versionado (a versão e a data ficam gravadas) e a gestão de consentimentos de localização e de condição de saúde. Verificar: Figura 12.

Configurações inseguras no Android e na página. O Android permitia cópia de segurança por adb e tráfego sem criptografia, e a página não tinha política de conteúdo. Foram desativados *allowBackup* e *usesCleartextTraffic* e adicionada uma política de segurança de conteúdo.

Texto livre sem tratamento (XSS). A entrada era gravada como digitada. Agora toda entrada perde sinais de menor e maior e caracteres de controle e tem tamanho limitado, e toda saída passa por escape antes de ir para a página. Os campos do Firestore não usam SQL; as consultas nunca são montadas com texto do usuário, o que afasta a injeção.

CPF e CNPJ sem validação e campos sem limite. Era possível digitar qualquer coisa e textos de tamanho infinito. Agora CPF e CNPJ têm máscara e dígitos verificadores conferidos, o telefone é conferido, todo campo tem limite de caracteres e as regras do Firestore repetem os limites de tamanho. Verificar: Figura 3.

Profissionais viam outros profissionais. As regras permitiam a um profissional ler a lista de todos os outros. Agora só clientes e o próprio dono leem perfis de profissionais.

Assistente de IA ignorava o filtro de região. O chat mostrava barbeiros que a tela não mostrava. Agora o assistente usa a mesma lista filtrada pela região do cliente, e a tela tem o botão "Ver todas as regiões".

Segredo de terceiros exposto em conversa. Uma chave de API do provedor de IA foi colada fora do código durante o desenvolvimento. Ela deve ser revogada no painel do provedor e substituída por outra, guardada apenas no arquivo *.env* do backend, que está no *.gitignore*. A varredura da Figura 16 confirma que o repositório não contém chaves.

3.2 Pendências abertas (assumidas)

Assistente sem verificação de login. O endereço */api/chat* aceita pedidos de quem conhecer a URL; há limites por IP, teto diário e moderação, mas falta exigir o token do Firebase. **Preço calculado no aparelho.** Um usuário técnico poderia gravar um preço diferente; a correção é calcular o valor em uma Cloud Function. **Dependências vulneráveis.** O *npm audit* aponta 11 vulnerabilidades no serviço de segurança (4 altas) e 2 no assistente (altas), todas com correção por atualização de versão principal (Figura 17). **MFA opcional.** O segundo fator é opcional por conta, e o MFA do Firebase exige o Identity Platform habilitado no console. **Política de conteúdo.** Ainda permite scripts embutidos. **Encarregado de dados.** Falta designar o encarregado e preencher o contato da política. **Regras do Firestore.** Não foram executadas no emulador. **Firebase App Check.** Não está ligado, então programas que não são o app ainda podem chamar o Firebase com um token válido.

4. Implementação de cada requisito

4.1 Criptografia, hash e sanitização

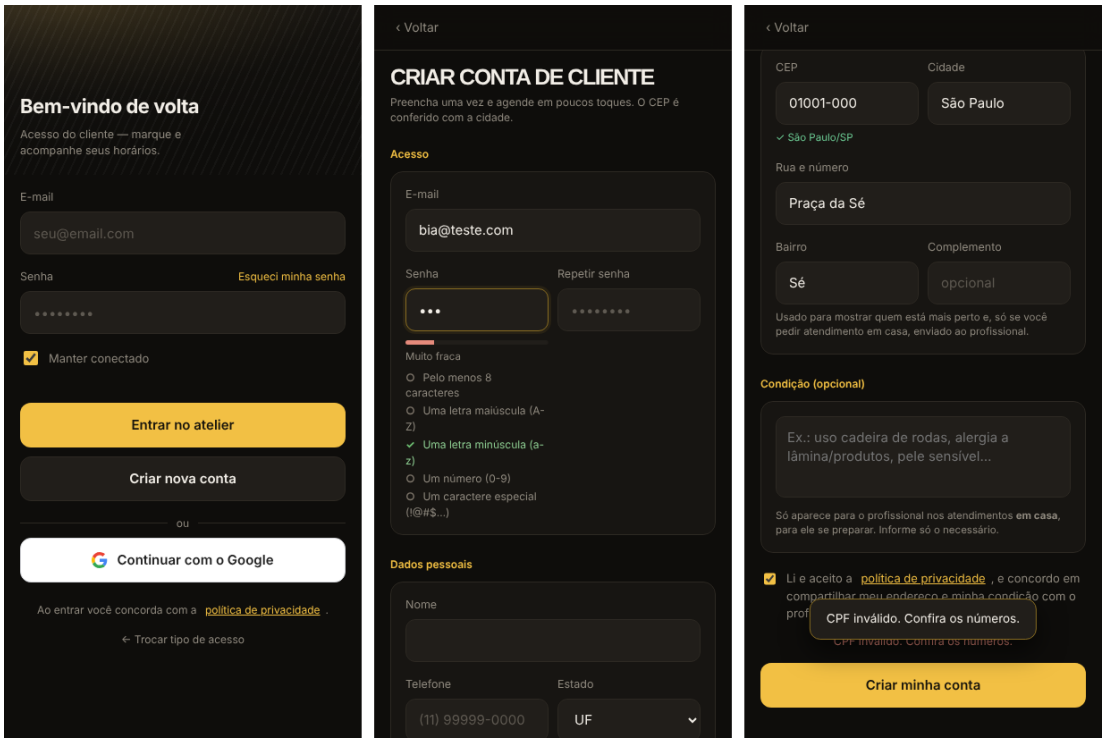
Hash de senha. As senhas ficam com o Firebase Authentication, que usa scrypt; o grupo nunca as vê. O bcrypt (custo 10) é usado no serviço de segurança para o código OTP e para o código de recuperação do servidor, que nunca são guardados em texto puro (arquivo *backend-seguranca/src/cripto.js*).

Criptografia. O serviço de segurança usa AES-256-GCM (arquivo *cripto.js*) para o backup, e o teste prova que adulterar o arquivo é detectado. No aplicativo, o cofre do código de recuperação é feito

com AES-256-GCM e uma chave derivada do código por PBKDF2-SHA256 com 600 mil iterações (arquivo *www/recuperacao.js*); o Firestore só recebe o texto cifrado. Em trânsito, tudo usa HTTPS e o Android recusa tráfego sem criptografia; em repouso, o Firestore criptografa por padrão. O CPF não tem criptografia de campo, o que está no plano de evolução.

Sanitização. A função *limpar* (arquivo *www/seguranca.js*) remove caracteres de controle e os sinais de menor e maior e limita o tamanho; a função *esc* escapa a saída; os servidores validam o corpo com a biblioteca zod; as regras do Firestore limitam o tamanho dos campos. O Firestore não usa SQL, de modo que a injeção de SQL não se aplica; o equivalente seria a injeção de operadores, evitada porque nenhuma consulta é montada com texto livre.

Figura 3 – Login com Google, requisitos de senha e recusa de CPF inválido

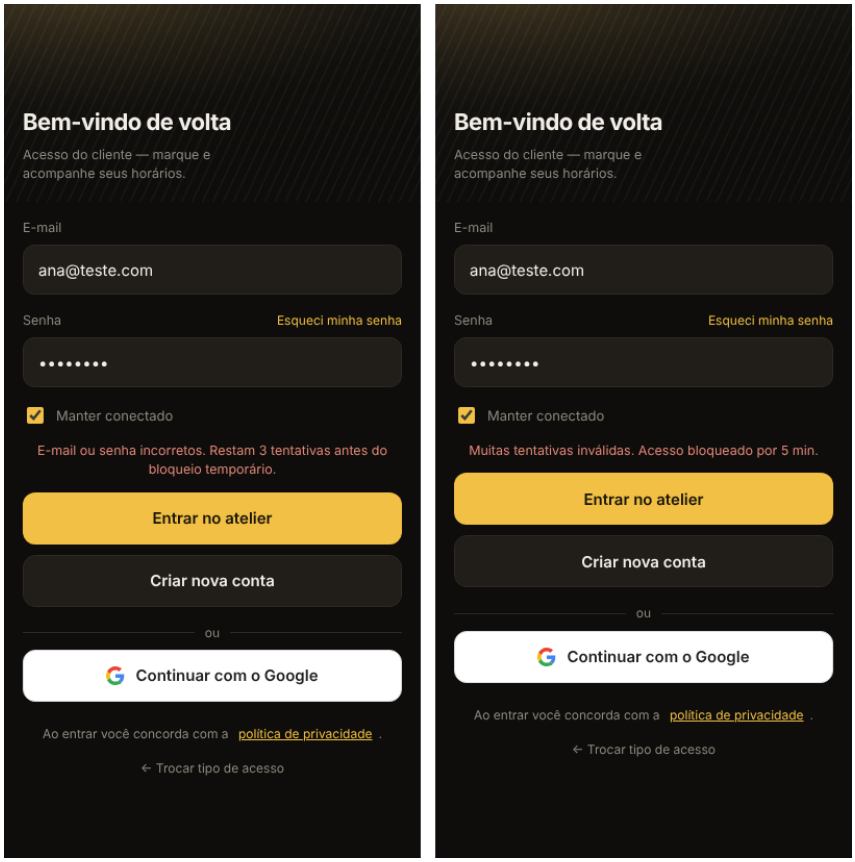


Fonte: ambiente de teste simulado do projeto, executado em 25/10/2026.

4.2 Rate limiting, sessão e tentativas de login

O serviço de segurança limita 120 pedidos por minuto por IP e limites mais estritos para login, OTP e recuperação (biblioteca express-rate-limit), além de contar falhas por e-mail; o assistente limita 10 por minuto e 80 por hora por IP, com teto diário. No aplicativo, cinco senhas erradas bloqueiam o login (5, 15 e 60 minutos) e a sessão expira por inatividade em 15 minutos (valor *inatividadeMin* em *www/seguranca-config.js*). O bloqueio local pode ser contornado por quem reinstalar o app; o do servidor (15 minutos) vale para qualquer aparelho.

Figura 4 – Mensagem genérica de erro e bloqueio temporário após cinco tentativas inválidas



Fonte: ambiente de teste simulado do projeto, executado em 25/10/2026.

Figura 5 – Cabeçalhos de segurança, origem não autorizada recusada (403), entrada inválida e limite de requisições (12 de 130 recusadas com 429)

```
bash — servidor de segurança em ambiente simulado

$ curl -i http://localhost:3199/health
HTTP/1.1 200 OK
Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https: data:;form-action 'self';fra
me-ancestors 'self';img-src 'self' data:;object-src 'none';script-src 'self';script-src-attr 'none';style-src
'self' https: 'unsafe-inline';upgrade-insecure-requests
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=15552000; includeSubDomains
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Vary: Origin
Content-Type: application/json; charset=utf-8
Content-Length: 11

{"ok":true}

$ curl -s -X POST http://localhost:3199/api/seg/pode-entrar -H "Origin: https://site-malicioso.example" -d "{}"
-H "Content-Type: application/json" -i | head -4
HTTP/1.1 403 Forbidden
Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https: data:;form-action 'self';fra
me-ancestors 'self';img-src 'self' data:;object-src 'none';script-src 'self';script-src-attr 'none';style-src
'self' https: 'unsafe-inline';upgrade-insecure-requests
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin

$ curl -s -X POST .../api/seg/pode-entrar -d '{"email":"nao-e-email"}'   # entrada inválida
{"erro":"Dados inválidos."}

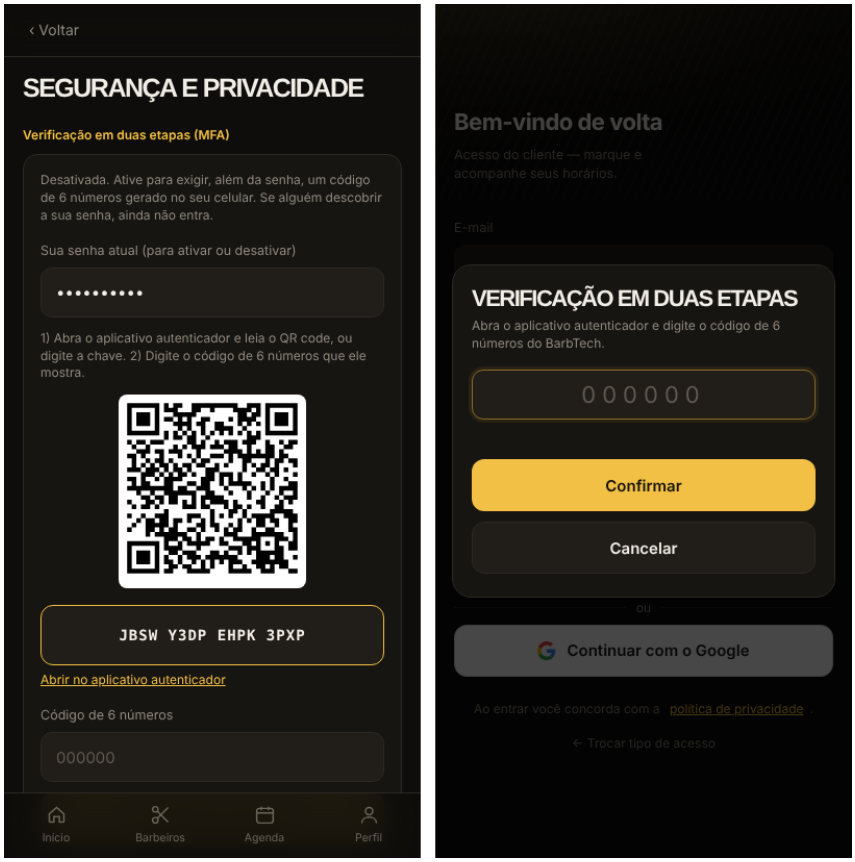
$ for i in $(seq 1 130); do curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3199/health; done | sort
| uniq -c
    118 200
     12 429
```

Fonte: ambiente de teste simulado do projeto, executado em 25/10/2026.

4.3 Senha forte e MFA

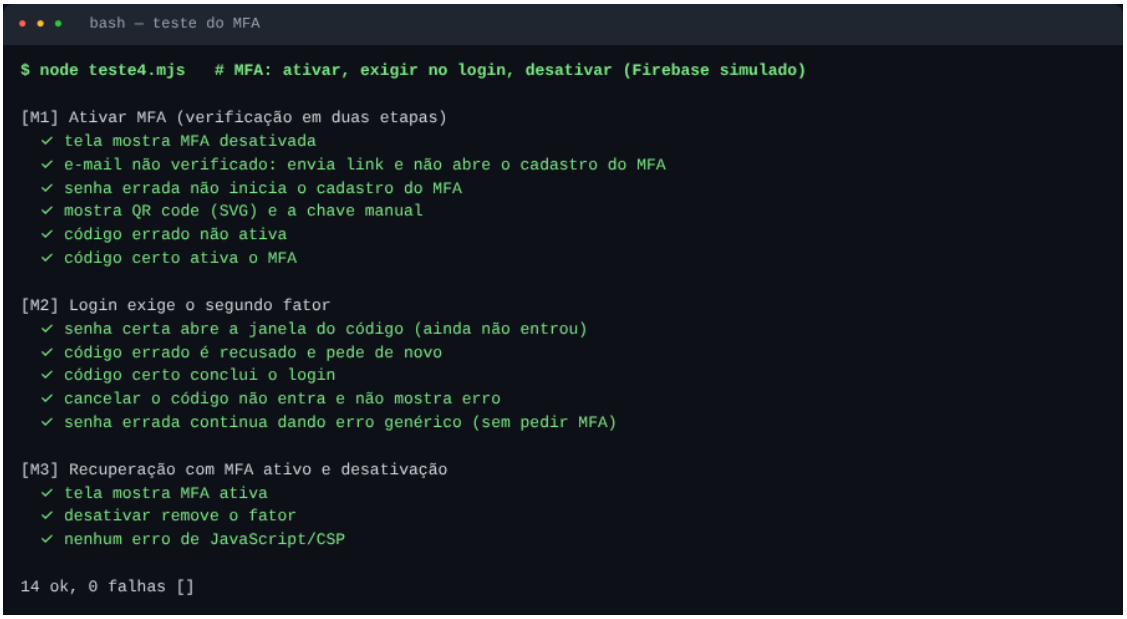
O validador de senha aparece no cadastro, na recuperação e na redefinição: uma barra de força e a lista de requisitos (8 ou mais caracteres, maiúscula, minúscula, número e caractere especial) que ficam verdes à medida que são atendidos (Figuras 3 e 8). O MFA usa o recurso nativo do Firebase Authentication: em Perfil → Segurança e privacidade, o usuário informa a senha, lê o QR code no Google Authenticator (ou digita a chave) e confirma com o primeiro código. Depois disso, o login só termina com o código de 6 números. Como é o servidor do Google que exige o segundo fator, alterar o aplicativo não o contorna. Para ativar, o e-mail precisa estar verificado (o app envia o link de verificação) e o projeto precisa ter o Identity Platform e o TOTP habilitados no console do Firebase. Quem entra pelo Google herda a verificação em duas etapas da própria conta Google.

Figura 6 – Ativação do MFA (QR code e chave) e janela de código no login



Fonte: ambiente de teste simulado do projeto, executado em 25/10/2026.

Figura 7 – Teste do MFA: ativar, exigir no login, recusar código errado, cancelar e desativar

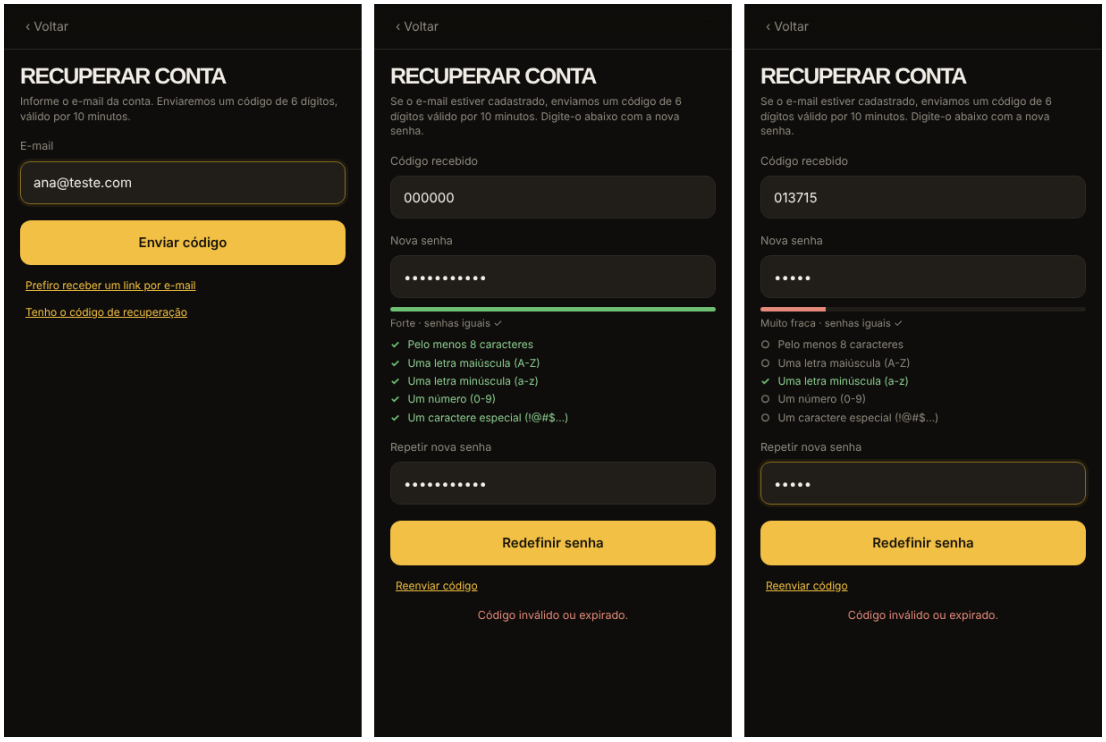


Fonte: teste executado com um módulo simulado do MFA do Firebase; o servidor real do Google não foi usado.

4.4 Recuperação de senha por OTP

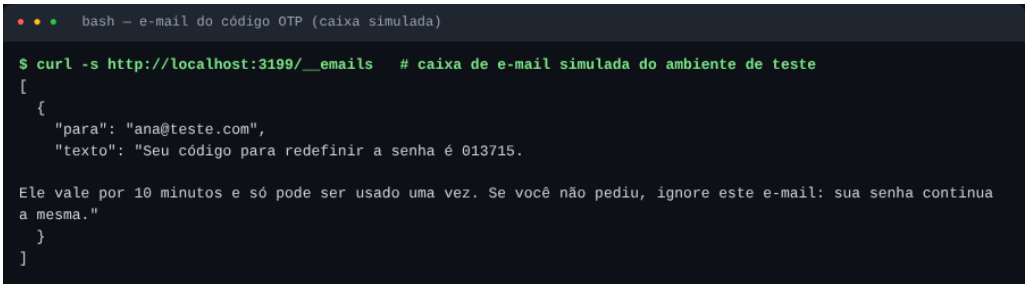
Em "Esqueci minha senha" → "Recuperar por e-mail", o usuário informa o e-mail e o serviço de segurança envia um código de 6 dígitos. O código vale 10 minutos, só pode ser usado uma vez e aceita 5 tentativas; guarda-se apenas o hash bcrypt. A resposta é a mesma exista o e-mail ou não, para não revelar quem tem conta. A nova senha precisa atender à política, e ao redefinir o serviço revoga as sessões abertas. Para a demonstração sem provedor de e-mail, o serviço aceita *SMTP_URL=console*, que escreve o e-mail no terminal. Esse modo nunca deve ser usado em produção.

Figura 8 – Recuperação por OTP: pedido do código, código errado recusado e nova senha com validador



Fonte: ambiente de teste simulado do projeto, executado em 25/10/2026.

Figura 9 – E-mail com o código OTP recebido (caixa simulada do ambiente de teste)

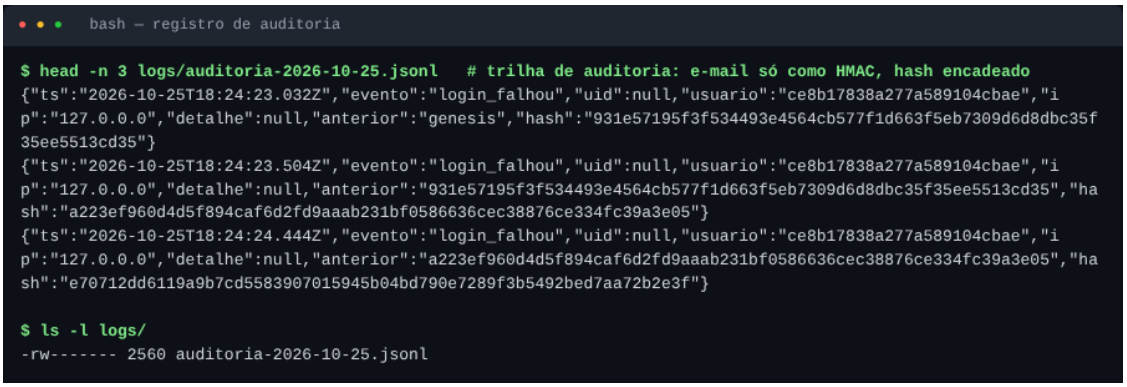


Fonte: ambiente de teste simulado do projeto, executado em 25/10/2026.

4.5 Logs, backup e hardening

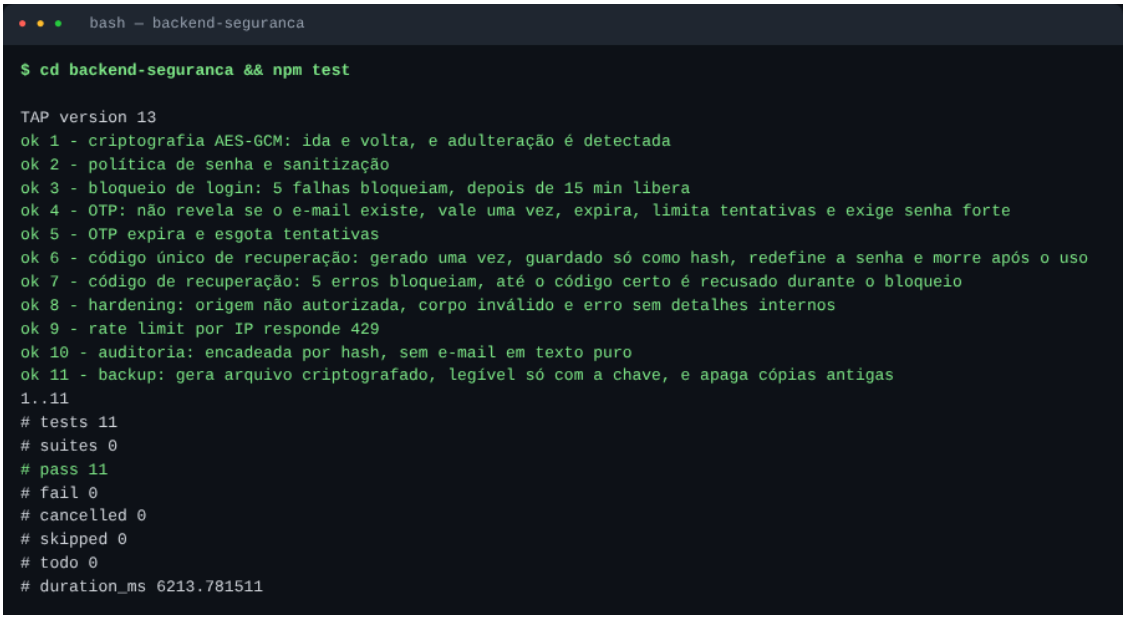
Logs. Cada evento (login com falha, bloqueio, OTP, redefinição, exclusão de conta) vira um registro de auditoria no Firestore e em arquivo. O e-mail é substituído por um HMAC, o IP é mascarado e cada registro carrega o hash do anterior: alterar uma linha quebra as seguintes (Figura 10). **Backup.** A rotina diária (*scripts/backup.js*) exporta as coleções, cifra com AES-256-GCM e apaga cópias com mais de 14 dias; o teste prova que só a chave abre o arquivo (Figura 11). **Hardening.** Os servidores não iniciam sem segredos (nenhum valor padrão), usam helmet, CORS por lista, corpo limitado a 8 ou 64 kilobytes, validação com zod e respondem erro genérico; no Android, backup e tráfego sem criptografia estão desativados; as regras do Firestore negam tudo que não está liberado. As limitações são que o backup fica no mesmo servidor (copiar para fora é pendência) e que a auditoria só existe quando o serviço de segurança está no ar.

Figura 10 – Registro de auditoria com hash encadeado, e-mail pseudonimizado e IP mascarado



Fonte: ambiente de teste simulado do projeto, executado em 25/10/2026.

Figura 11 – Os 11 testes automatizados do serviço de segurança (criptografia, bloqueio, OTP, hardening, rate limit, auditoria e backup)



Fonte: ambiente de teste simulado do projeto, executado em 25/10/2026.

4.6 Tratamento genérico de erros

A função *msgErro* (*www/app.js*) converte qualquer exceção em uma frase em português escrita pelo grupo, e o detalhe técnico vai apenas para o console do desenvolvedor. Os servidores devolvem *{"erro": "Dados inválidos."}* ou erro genérico, sem pilha de execução, e o teste da Figura 5 confirma isso. O teste ponta a ponta injeta um erro técnico do Firebase e confirma que o texto não chega à tela (Figura 14).

4.7 Conformidade com a LGPD

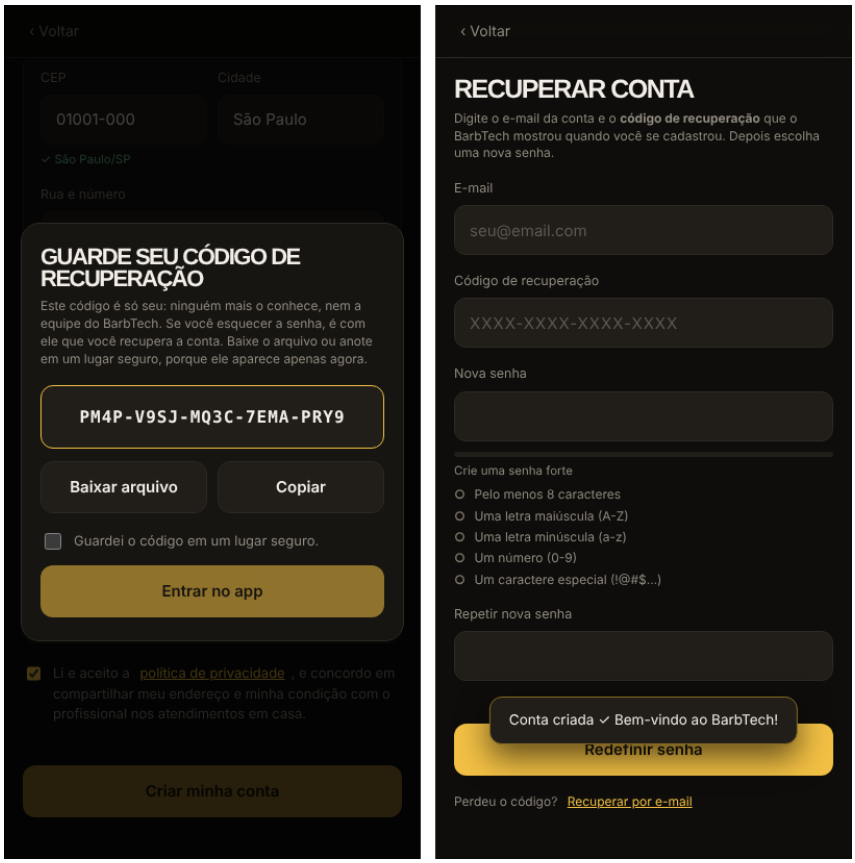
A política de privacidade abre sem login e explica quem trata os dados, quais dados, finalidade e base legal, com quem são compartilhados, por quanto tempo, como são protegidos, os direitos do titular, a regra para menores e as mudanças da política. O consentimento é gravado com a versão e a data; se a política muda, o app pede o aceite de novo. Em Perfil → Segurança e privacidade, o titular liga e desliga o uso da localização e o compartilhamento da condição de saúde, baixa uma cópia dos dados e exclui a conta com os dados. O contato do responsável é definido em *www/seguranca-config.js* e hoje está vazio, o que é pendência (a tela mostra um texto de lembrete).

Figura 12 – Segurança e privacidade (consentimentos e direitos) e política de privacidade



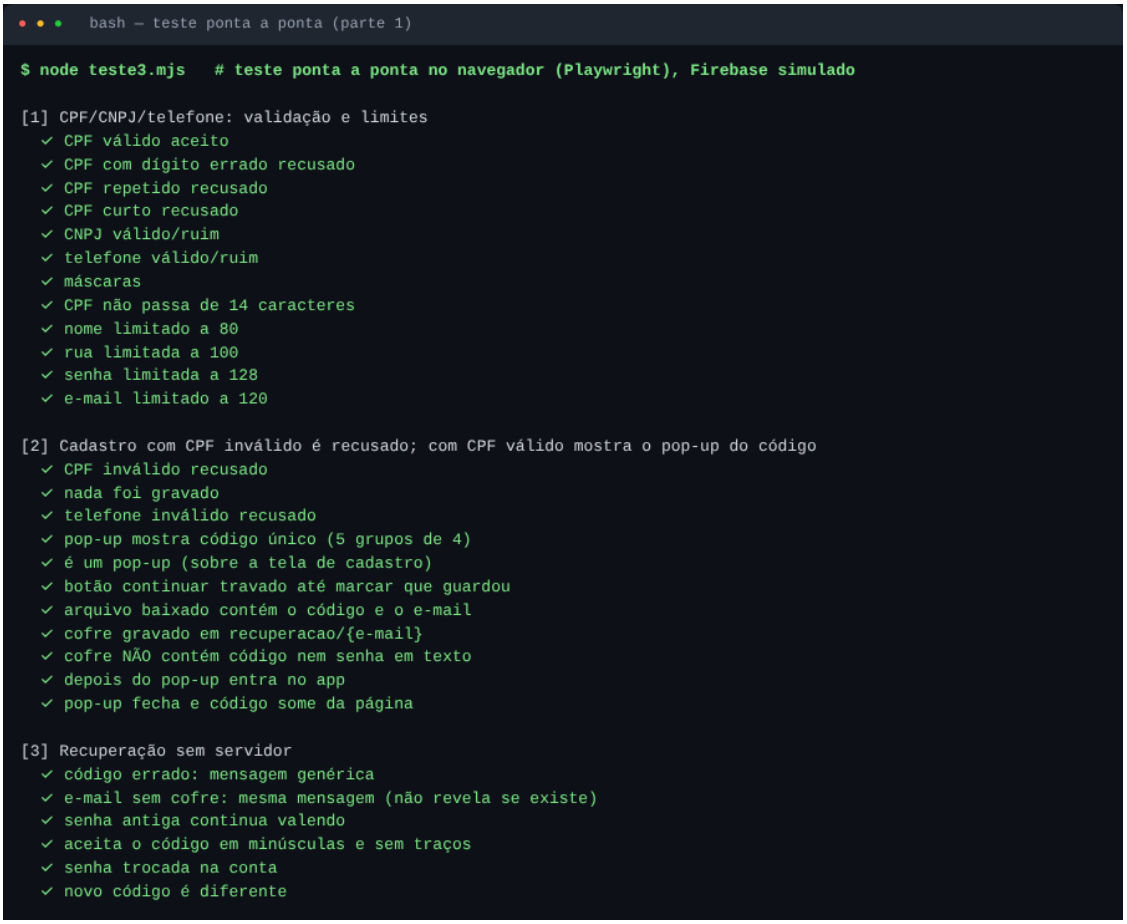
Fonte: ambiente de teste simulado do projeto, executado em 25/10/2026.

Figura 13 – Código de recuperação exibido no cadastro e tela de recuperação de conta



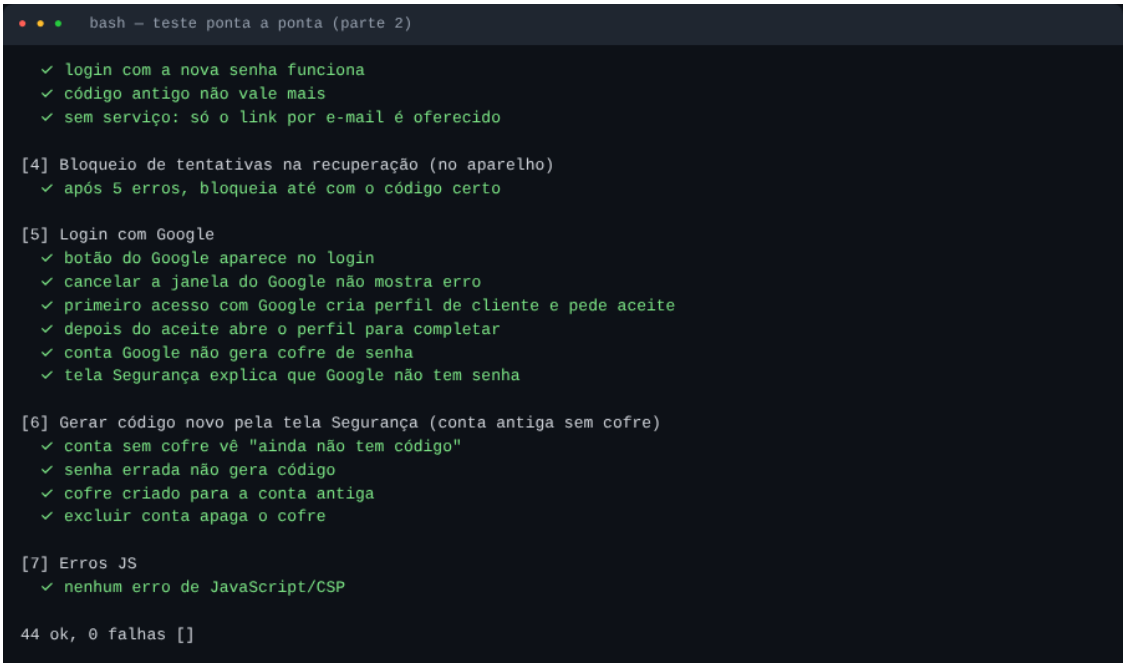
Fonte: ambiente de teste simulado do projeto, executado em 25/10/2026.

Figura 14 – Teste ponta a ponta, parte 1



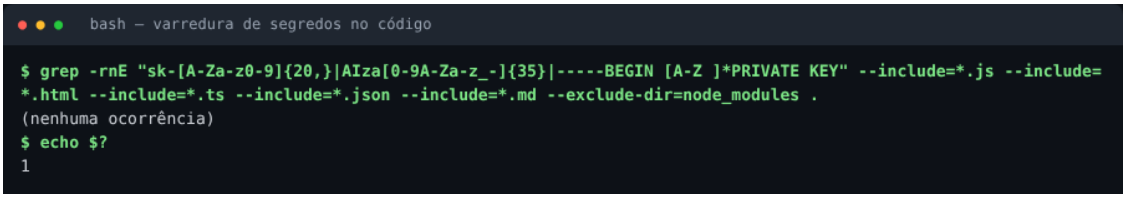
Fonte: ambiente de teste simulado do projeto, executado em 25/10/2026.

Figura 15 – Teste ponta a ponta, parte 2 (44 verificações aprovadas)



Fonte: ambiente de teste simulado do projeto, executado em 25/10/2026.

Figura 16 – Varredura de segredos no repositório: nenhuma chave encontrada



Fonte: ambiente de teste simulado do projeto, executado em 25/10/2026.

Figura 17 – Auditoria de dependências: 11 vulnerabilidades no serviço de segurança e 2 no assistente



Fonte: ambiente de teste simulado do projeto, executado em 25/10/2026.

5. Plano de evolução de maturidade e novas tecnologias

O grupo usa o OWASP SAMM como régua: hoje o projeto está no nível 1 (práticas básicas e pontuais) e a meta é o nível 2 (práticas repetíveis e verificáveis). O plano está em três horizontes.

5.1 Curto prazo (até o próximo semestre)

Atualizar as dependências apontadas pelo *npm audit* e tornar a auditoria uma etapa obrigatória a cada entrega. Exigir o token do Firebase no assistente e limitar o uso por usuário. Calcular o preço do agendamento em uma Cloud Function. Executar as regras do Firestore no emulador, com testes automatizados que provem o que cada papel pode ler e escrever. Ligar o **Firebase App Check** com o Play Integrity, para que só o app autêntico fale com o Firebase. Designar o encarregado de dados e preencher o contato da política. Remover os scripts embutidos da página e endurecer a política de conteúdo. Tornar o MFA obrigatório para profissionais.

5.2 Médio prazo

Introduzir **passkeys** (WebAuthn/FIDO2), que eliminam a senha e resistem a phishing, em complemento ao MFA por aplicativo. Adicionar verificação de senhas contra listas de vazamentos. Criar um pipeline de integração contínua com análise estática (CodeQL ou Semgrep), varredura de dependências automática (Dependabot) e varredura de segredos, além de uma lista de materiais de software (SBOM). Fazer testes dinâmicos com o OWASP ZAP contra os servidores. Criptografar por campo o CPF e o CNPJ com chaves em um serviço de gestão de chaves (Cloud KMS). Copiar o backup para um armazenamento externo, com teste periódico de restauração. Publicar os servidores atrás de um proxy reverso com TLS e de um WAF, com alertas de custo e de cota. Desbloqueio por biometria ao reabrir o app e fixação de certificado (*certificate pinning*).

5.3 Longo prazo

Formalizar a governança: inventário das operações de tratamento, relatório de impacto à proteção de dados (RIPD), plano de resposta a incidentes com simulação anual e canal de reporte de vulnerabilidades (*security.txt*). Contratar um teste de invasão externo e considerar um programa de recompensas. Avaliar a trilha para as normas ISO/IEC 27001 e 27701. Adotar o modelo de confiança zero entre os serviços (identidades de serviço, rotação automática de segredos em um cofre) e um monitoramento central de logs com alertas (SIEM). Rever o modelo de ameaças a cada funcionalidade nova, como pagamentos ou chat entre usuários, que trazem riscos que hoje não existem.

6. Roteiro da demonstração presencial

O roteiro abaixo prova cada proteção em ordem, com o que dizer e o que mostrar. Prepare antes: abra o app (*npx serve www* na pasta do projeto, ou o aplicativo instalado), suba o serviço de segurança (*cd backend-seguranca*, copie *.env.example* para *.env* com *SMTP_URL=console*, coloque o arquivo da conta de serviço do Firebase, rode *npm install* e *npm start*) e aponte *www/seguranca-config.js* para *http://localhost:3100* (e o endereço do app em *ALLOWED_ORIGINS*). Para a inatividade, troque *inatividadeMin* por 0.5.

1. Threat modeling (2 min). Mostre as Figuras 1 e 2 e explique as fronteiras de confiança e os onze riscos. **2. Senha forte e CPF (2 min).** No cadastro, digite uma senha fraca e mostre o validador ficando verde; digite um CPF inválido e mostre a recusa. **3. Código único (1 min).** Conclua o cadastro e mostre a janela do código com os botões de baixar e copiar. **4. Erro genérico e bloqueio (2 min).** Erre a senha cinco vezes: a mensagem é a mesma para qualquer erro e, na quinta, o acesso é bloqueado com o tempo restante. **5. OTP (3 min).** Em "Esqueci minha senha" → "Recuperar por e-mail", mostre o terminal do servidor com o e-mail e o código, digite um código errado (recusa), depois o certo com uma senha fraca (recusa) e uma forte (aceita); entre com a senha nova. **6. MFA (3 min).** Em Segurança e privacidade, ative o MFA com o Google Authenticator; saia e entre de novo: a senha sozinha não basta. **7. Google (1 min).** Mostre o botão de login com Google. **8. LGPD (2 min).** Abra a política sem login, mostre o consentimento, baixe os dados e mostre o botão de excluir conta. **9. Sessão (1 min).** Deixe o app parado até o aviso e a saída automática.

10. Servidor (3 min), no terminal: *curl -i http://localhost:3100/health* mostra os cabeçalhos; *for i in $(seq 1 130); do curl -s -o /dev/null -w "%{http_code}\n" http://localhost:3100/health; done | sort | uniq -c* mostra os pedidos recusados com 429; um *curl* com o cabeçalho *Origin* de outro site mostra o 403; *cd backend-seguranca && npm test* roda os 11 testes; *head -n 3 logs/auditoria-*.jsonl* mostra a auditoria; *npm run backup* e *ls -l backups* mostram o backup cifrado; e *npm audit* mostra, com honestidade, as dependências a atualizar. **11. Fechamento (1 min).** Apresente a seção 3.2 (o que ainda falta) e o plano de evolução: mostrar o que se sabe que falta é parte da maturidade de segurança.

7. Limitações desta entrega

Os testes usaram o Firebase simulado e, portanto, não provam o comportamento do Firebase real: o MFA depende do Identity Platform e do TOTP habilitados; o login com Google depende do provedor ativado, do domínio autorizado e, no Android, do SHA-1 e do plugin nativo; o envio de e-mail depende de um provedor SMTP. As regras do Firestore não foram testadas no emulador. Várias proteções do aplicativo (política de senha, bloqueio local, sessão por inatividade e limites de campo) rodam no aparelho e só valem do lado do servidor onde o texto diz expressamente. As pontuações de risco e de CVSS do relatório são estimativas do grupo.